DAA Lab 1
Complexity Comparison

The following table follows the complexity summary specified in the assignment and the implementations in the supplied notebook.

Algorithm	Best Case	Average Case	Worst Case	Space
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n) avg.; O(n) worst*
Recursive Fibonacci	O(2■)	O(2■)	O(2■)	O(n)
Iterative Fibonacci	O(n)	O(n)	O(n)	O(1)
Dynamic Programming Fibonacci	O(n)	O(n)	O(n)	O(n)

*For Quick Sort, the assignment's table gives O(log n) stack space; with the required last-element pivot, the worst-case recursion depth can become O(n) on already sorted/reverse-sorted input.

Interpretation

Bubble Sort: Early exit gives O(n) on already sorted input, but reverse/random inputs can require quadratic work.

Insertion Sort: Performs well on small or nearly sorted inputs; reverse/random inputs can require O(n²) work.

Merge Sort: Maintains O(n log n) time across cases and is therefore predictable for large inputs, at the cost of O(n) auxiliary space.

Quick Sort: Average performance is O(n log n), but the required last-element pivot creates an O(n²) worst case on sorted/reverse-sorted data, matching the experimental results.

Recursive Fibonacci: Recomputes the same subproblems repeatedly, giving exponential running time and linear recursion-stack space.

Iterative Fibonacci: Uses a single loop and two variables, giving O(n) time and O(1) auxiliary space.

Dynamic Programming Fibonacci: Stores previously computed values, reducing time to O(n) while using O(n) memory.

Theoretical vs Practical Performance

The measurements strongly reflect the theoretical trends. The clearest examples are Quick Sort's huge slowdown on ordered inputs with a last-element pivot and Recursive Fibonacci's jump to 20.2445 seconds at n=40. Small-input timings also contain constant overhead and measurement variability, so they should not be interpreted as exact Big-O coefficients.